

Resumen Teorico – Practico 5

Testing basado en expresiones logicas

Predicados, clausulas y cobertura logica

Testing de Software 2024

Índice

Introduccion	3
1. Predicados y clausulas	3
1.1. Definiciones basicas	3
1.2. ¿De donde salen los predicados?	4
1.3. Traduccion de lenguaje natural	4
1.4. Datos empiricos	4
1.5. Expresiones logicas desde el codigo	5
2. Notacion estandar	5
3. Cobertura de predicados y de clausulas	5
3.1. Ejemplo: $p = a \vee (b \wedge c)$	6
3.2. Limitacion practica de CC	6
4. Cobertura combinatoria	6
5. Clausulas activas	7
5.1. Determinacion	7
5.2. Active Clause Coverage (ACC)	7
5.3. GACC, CACC y RACC	8
5.4. Ejemplo: $p = a \wedge (b \vee c)$	8
6. Cobertura inactiva	8

7. Mapa de subsuncion	9
8. Origenes de las expresiones logicas	9
8.1. Desde el codigo fuente	9
8.2. Desde especificaciones	10
8.3. Desde maquinas de estados	10
8.4. Desde lenguaje natural	10
9. Conexion con MCDC	10
10.Como derivar tests con cobertura logica	11
10.1. Receta paso a paso	11
10.2. De clausulas a inputs	11
11.Pequeno glosario	11
Conclusion	11

Introduccion

Este resumen sintetiza el material teorico asociado al TP5:

- *Introduction to Software Testing* (Ammann & Offutt, 2da edicion), Capitulo 8: **Logic Coverage**.
- *Notas 09: Cobertura logica* (V. Bengolea, primera parte).
- *Notas 09: Cobertura logica* (V. Bengolea, segunda parte).

Idea general. El software contiene constantemente *expresiones logicas*: las guardas de un `if` o `while`, las transiciones de una FSM, las condiciones mencionadas en una especificacion o un requisito. Una expresion logica con n variables booleanas tiene 2^n combinaciones; en la practica es inviable probarlas todas. Los **criterios de cobertura logica** eligen un subconjunto pequeno y representativo, atando los tests a la estructura de la formula.

Idea clave

Los criterios de cobertura logica responden a una pregunta operativa: “si una decision booleana toma muchos valores distintos, ¿cuales necesito probar para estar razonablemente seguro de que la condicion fue codificada bien?”.

1. Predicados y clausulas

1.1. Definiciones basicas

Definicion

Predicado. Expresion que se evalua a un valor booleano. Un predicado puede contener:

- variables booleanas,
- variables no booleanas combinadas con operadores relacionales ($<$, $=$, $!$, $\dots$),
- llamadas a funciones que devuelven un booleano.

La estructura interna se forma con los operadores logicos $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\oplus$, $\leftrightarrow$.

Clausula. Predicado que *no contiene operadores logicos*. En la formula $((a > b) \vee C) \wedge p(x)$ las clausulas son $(a > b)$, C y $p(x)$.

Idea clave

Las *clausulas* son los “atomos” booleanos de una formula: son lo mas pequeno que aun se evalua a *true* o *false*. Los criterios actuan sobre clausulas porque cada clausula corresponde, conceptualmente, a una decision atomica que el programador codifica.

1.2. ¿De donde salen los predicados?

Las expresiones logicas no aparecen solo en el codigo. Surgen tambien de:

- **Codigo:** guardas de `if/while`, condiciones de salida de bucles, cortocircuitos.
- **Maquinas de estados y statecharts:** las transiciones se etiquetan con guardas booleanas.
- **Requisitos y especificaciones** (formales o informales). Por ejemplo, las precondiciones de un metodo: “the stack is not full and o is not null” se traduce a $\neg this.isFull() \wedge o \neq null$.
- **Lenguaje natural**, convirtiendo enunciados a logica formal.
- **Consultas SQL** y otras DSL que evaluan condiciones.

La FAA estadounidense exige cobertura de expresiones logicas (concretamente MCDC, un primo hermano de CACC) en *safety-critical software*.

1.3. Traduccion de lenguaje natural

Pasar de una frase a una formula logica es una fuente clasica de errores:

- “*I am interested in SWE 637 and CS 652*”: la conjuncion natural es realmente un *O* logico: $course = swe637 \vee course = cs652$.
- “*If you leave before 6:30, take Braddock; if you leave after 7:00, take Prosperity*”: la formula directa $(t < 6:30 \rightarrow Braddock) \wedge (t > 7:00 \rightarrow Prosperity)$ es incompleta: deja sin cubrir el intervalo $[6:30, 7:00]$. Una version corregida es $(t < 6:30 \rightarrow Braddock) \wedge (t \geq 6:30 \rightarrow Prosperity)$.

Idea clave

“**Y**” del castellano a menudo se traduce a *O logico*; “**O**” del castellano suele significar XOR (“café o té”); las descripciones por casos suelen tener *huecos* no cubiertos. Antes de aplicar criterios de cobertura conviene asegurarse de que la traduccion sea fiel.

1.4. Datos empiricos

Estudio sobre 63 programas open source y mas de 400 000 predicados:

- 88,5% tiene una sola clausula.
- 9,5% tiene dos.

- 1,35% tiene tres.
- 0,65% tiene cuatro o mas.

Es decir, los predicados con muchas clausulas son la minoria, pero son justamente donde la diferencia entre criterios (PC, CC, CACC, ...) se vuelve interesante.

1.5. Expresiones logicas desde el codigo

Una decision como

```
if (((a > b) || C) && (x < y)) o.m(); else o.n();
```

produce el predicado $p = ((a > b) \vee C) \wedge (x < y)$ con clausulas $(a > b)$, C y $(x < y)$.

2. Notacion estandar

Definicion

A lo largo del capitulo se usa la siguiente notacion:

- P : conjunto de predicados.
- p : un predicado de P .
- C : conjunto de clausulas en P .
- C_p : conjunto de clausulas del predicado p .
- c : una clausula.
- RT : conjunto de requisitos de test generados por el criterio.

3. Cobertura de predicados y de clausulas

Los dos criterios mas simples piden que cada predicado o cada clausula tome ambos valores de verdad.

Definicion

Predicate Coverage (PC). Para cada predicado $p \in P$, RT contiene dos requisitos: p se evalua a *true* y p se evalua a *false*.

Definicion

Clause Coverage (CC). Para cada clausula $c \in C$, RT contiene dos requisitos: c se evalua a *true* y c se evalua a *false*.

Relacion con cobertura estructural. Si los predicados provienen de las decisiones del código, entonces:

- PC equivale a *cobertura de aristas* (Edge Coverage) del CFG: cada rama del `if` tiene que ejecutarse.
- CC equivale a *cobertura de condiciones*: cada condicion booleana individual debe evaluar a T y a F .

Idea clave

CC y PC no se subsumen entre si. PC ignora la estructura interna del predicado; CC ignora si el predicado toma realmente ambos valores. Pueden construirse suites que cumplan uno sin cumplir el otro.

3.1. Ejemplo: $p = a \vee (b \wedge c)$

Ejemplo

Suite $\{(T, F, F), (F, T, T)\}$:

- Cada clausula toma T y $F \Rightarrow$ CC ok.
- Pero p vale T en los dos casos $\Rightarrow$ PC *no* se cumple.

Suite $\{(T, T, T), (F, T, F)\}$:

- p toma T y $F \Rightarrow$ PC ok.
- La clausula b queda fija en $T \Rightarrow$ CC *no* se cumple.

3.2. Limitacion practica de CC

CC pide que cada clausula tome ambos valores, pero *sin coordinarse* con el predicado. En $p = a \wedge b$, los pares $(a = T, b = F)$ y $(a = F, b = T)$ satisfacen CC, pero ninguno ejerce el caso $p = T$. Por eso CC suele ir acompañado de PC o reemplazado por criterios mas finos como CACC.

4. Cobertura combinatoria

Definicion

Combinatorial Coverage (CoC). Para cada predicado $p \in P$, RT contiene un requisito por cada combinacion posible de valores de las clausulas de C_p .

CoC es “el criterio ideal”: cubrir toda la tabla de verdad. El costo es exponencial: con n clausulas hace falta 2^n tests. En la practica se reserva solo para predicados muy chicos.

Cuello de botella. CoC subsume PC y CC, pero suele ser inviable para $n \geq 4$ o cuando hay muchos predicados.

5. Clausulas activas

CC peca por desacoplado y CoC por excesivo. La idea intermedia es: para cada clausula, exigir casos en los que esa clausula realmente *determine* el valor del predicado.

5.1. Determinacion

Definicion

Una clausula $c_i \in C_p$ **determina** el predicado p si, al fijar las demas clausulas (las menores) y cambiar c_i , el valor de p cambia. De forma equivalente, p se vuelve una funcion de c_i solo: $p|_{c_i=T} \neq p|_{c_i=F}$.

Ejemplo

En $p = a \vee b$, la clausula a determina cuando $b = F$ (sino el OR siempre es T). En $p = a \wedge b$, a determina cuando $b = T$ (sino el AND siempre es F).

5.2. Active Clause Coverage (ACC)

Definicion

Active Clause Coverage (ACC). Para cada $p \in P$ y cada clausula $c_i \in C_p$ (la *clausula mayor*), elegir valores para las clausulas menores c_j ($j \neq i$) tales que c_i determina p . RT contiene dos requisitos por clausula mayor:

1. $c_i = T$ con las menores fijadas asi;
2. $c_i = F$ con las menores fijadas asi.

ACC se queda ambiguo porque no precisa *como* se eligen las clausulas menores. De ahi nacen tres variantes que se diferencian en la rigidez de esa eleccion.

5.3. GACC, CACC y RACC

Definicion

General Active Clause Coverage (GACC). Para cada c_i , las menores pueden tener cualquier valor mientras c_i determine p . No se exige nada sobre el valor de p entre los dos tests.

Correlated Active Clause Coverage (CACC). Igual que GACC, pero *ademas* se exige que en el par de tests p tome ambos valores (T y F): la clausula mayor debe *causar correlativamente* el cambio.

Restricted Active Clause Coverage (RACC). Igual que CACC, pero las clausulas menores deben tener *exactamente los mismos valores* en los dos tests del par.

Idea clave

De debil a fuerte: $GACC \prec CACC \prec RACC$. CACC es el mas usado en la practica; es esencialmente lo que la industria llama **MCDC** (Modified Condition / Decision Coverage). RACC es mas exigente y a veces *infeasible* (no existen asignaciones validas que dejen las menores fijas y permitan que la mayor determine).

5.4. Ejemplo: $p = a \wedge (b \vee c)$

Ejemplo

Mayor a. a determina cuando $b \vee c = T$. Par CACC: $(T, T, F) \Rightarrow p = T$ vs $(F, T, F) \Rightarrow p = F$.

Mayor b. b determina cuando $c = F$ y $a = T$. Par CACC: $(T, T, F) \Rightarrow p = T$ vs $(T, F, F) \Rightarrow p = F$.

Mayor c. c determina cuando $b = F$ y $a = T$. Par CACC: $(T, F, T) \Rightarrow p = T$ vs $(T, F, F) \Rightarrow p = F$.

Las menores estan fijadas a un unico valor por par, asi que esta suite es RACC tambien.

6. Cobertura inactiva

La idea simetrica: para cada clausula mayor, mantenerla en cada valor mientras *no* determina el predicado, y mostrar que en ese caso p no depende de c_i .

Definicion

Inactive Clause Coverage (ICC). Para cada c_i se eligen menores que hagan que c_i no determine p , y se exigen cuatro requisitos por clausula mayor:

1. $c_i = T, p = T$;
2. $c_i = T, p = F$;
3. $c_i = F, p = T$;
4. $c_i = F, p = F$.

GICC no restringe las menores; **RICC** las pide identicas en cada par.

ICC ayuda a detectar errores de tipo “clausula sobrante”: si el codigo agrega una clausula c_i que en la formula correcta no aparece, ICC trata de exhibirlo verificando que p se mantiene igual cuando c_i cambia “por las razones correctas”.

7. Mapa de subsuncion

Las relaciones de subsuncion entre los criterios (un criterio X *subsume* a Y si toda suite que satisface X tambien satisface Y) son:

- $\text{CoC} \succ \text{CACC} \succ \text{GACC} \succ \text{CC}$.
- $\text{CACC} \succ \text{PC}$.
- $\text{RACC} \succ \text{CACC}$, pero RACC puede ser *infactible* en ciertos predicados.
- Ni CC subsume a PC , ni PC subsume a CC .

Idea clave

Una manera intuitiva de pensar la jerarquia:

- PC ve el predicado como una caja negra: “salgan ambos resultados”.
- CC mira las clausulas, pero sin coordinarlas con p .
- $\text{ACC}/\text{CACC}/\text{RACC}$ son lo que se quiere casi siempre: cada clausula tiene que importar.
- CoC es la cota superior brutal: probar todo.

8. Origenes de las expresiones logicas

8.1. Desde el codigo fuente

Una expresion booleana del codigo (sin importar como esta agrupada con parentesis) se mapea a una formula *con clausulas* en la que cada operando relacional o booleano

elemental es una clausula. El cortocircuito ($\&\&$, $\|\|$) en lenguajes como Java suele tratarse como evaluacion sintactica completa para fines del criterio.

8.2. Desde especificaciones

Las precondiciones y postcondiciones (formato JML, contratos, comentarios estilo Eiffel) contienen formulas logicas explicitas. Por ejemplo:

```
/*@precondition: the stack is not full and o object is not null;
   */
public void push(Object o);
```

se traduce a $\neg this.isFull() \wedge o \neq null$, con dos clausulas claras sobre las que aplicar PC/CC/CACC.

8.3. Desde maquinas de estados

Las transiciones de un statechart estan etiquetadas por *guardas* de la forma *trigger*[*guard*]/*accion*. La guarda es un predicado al que se le puede pedir cobertura logica: PC sobre transiciones, CACC sobre guardas con varias clausulas.

8.4. Desde lenguaje natural

Como se discutió, la traduccion es propensa a errores de cuantificadores, conectivas y casos no cubiertos. Antes de aplicar cobertura logica conviene formalizar y revisar la formula.

9. Conexion con MCDC

Cuando la industria (especialmente aviacion, DO-178B/C) habla de cobertura logica usualmente se refiere a **MCDC**: *Modified Condition/Decision Coverage*. Es esencialmente lo mismo que **CACC**: cada decision toma todos los valores; cada condicion toma todos los valores; y se demuestra que cada condicion influye de forma independiente en el resultado de la decision.

Idea clave

Si te piden “MCDC” en una norma como DO-178C, la receta operativa es la misma que para CACC: para cada clausula mayor, dos casos donde ella determina el predicado y donde el predicado cambia.

10. Como derivar tests con cobertura logica

10.1. Receta paso a paso

1. Identificar el predicado p y sus clausulas C_p .
2. Construir o pensar la tabla de verdad (o un subconjunto razonado de ella).
3. Elegir el criterio (PC, CC, CACC, RACC, ...) segun el nivel de exigencia.
4. Para cada requisito del criterio, encontrar valores de las clausulas que lo satisfagan.
5. Traducir cada combinacion de clausulas a inputs concretos del codigo (a veces hace falta invertir las relaciones $a = (x < y)$ para encontrar x, y).
6. Escribir los tests y verificar con la herramienta correspondiente.

10.2. De clausulas a inputs

Una clausula como $a := curTemp < dTemp - thresholdDiff$ es verdadera cuando se cumple la desigualdad. Para forzar $a = T$ se eligen valores numericos que la cumplan. Cuando varias clausulas comparten variables (e.g. `curTemp` aparece en a y c), hay que armar inputs que respeten todas las restricciones simultaneamente; a veces es necesario fijar parametros adicionales (`thresholdDiff`, `minLag`) para tener control fino.

11. Pequeno glosario

- **Clausula mayor.** La clausula sobre la que se esta razonando (la que “cambia”) en un par CACC/RACC.
- **Clausulas menores.** Las restantes; se eligen para que la mayor determine p .
- **Test factible.** Aquel para el que existe una entrada del programa que produzca esa combinacion de valores. Si la combinacion es contradictoria con la logica del codigo (e.g. dos clausulas se excluyen), el test es *infactible*.
- **Subsuncion.** Si X subsume a Y , todo conjunto de tests que satisface X satisface tambien Y .

Conclusion

La cobertura logica organiza, con definiciones claras, una intuicion vieja: “si una decision del programa tiene varias condiciones, hay que mover cada una y mirar como cambia el

resultado”. PC y CC son puntos de partida, CACC/RACC formalizan la idea de *causalidad*, y CoC actua como cota superior. En la practica, la combinacion mas usada es CACC (equivalente a MCDC), por ser exigente sin volverse exponencial.